Pretraining and Reasoning Fine-Tuning of Small Monolingual Indic Transformers: Hindi and Assamese

Consolidated Final Report (Phases 1, 2 and 3)

Shubhadeep Mandal (Roll No. 2025201056)
Branch `phase-3` • Language Models and Agents, Monsoon 2026
Higher-resource model H: Hindi (Devanagari) • Lower-resource model L: Assamese (Eastern Nagari, অসমীয়া)

Abstract. We built two independent monolingual pipelines, from corpus collection to reasoning fine-tuning, for Hindi and Assamese. Each language has its own 16,384-piece BPE tokenizer (fertility 1.28 and 1.43 tokens per word, no unknown tokens) and two 25M-parameter decoder-only Transformers written from PyTorch primitives: a Baseline V1 (learned positions, GELU, LayerNorm) and a Modern V2 (RoPE, SwiGLU, RMSNorm), each pretrained on 500M tokens. V2 reaches lower held-out perplexity in both languages (Hindi 36.15 vs 38.66; Assamese 51.55 vs 54.53 on 512-token windows), a 5–7% advantage. Fine-tuned on synthetic relational reasoning (44 runs, 2 seeds per recipe), the models learn the task and generalise to chains up to three times longer than any in training: Hindi V1 with Direct SFT scores 0.961 on the test set and 0.846 on 10-hop chains, against a majority baseline of 0.48. A leakage analysis shows the main test is structurally covered by training (a lookup of training answers scores 0.95), so the evidence of reasoning comes from deeper, structurally new splits where that lookup is at chance. Direct SFT beats Chain-of-Thought SFT, V1 beats V2 despite its higher perplexity, and Assamese trails Hindi (best 0.833). Every model is misled by premises about a different attribute. After fine-tuning, attention changes mostly in the late layers, where the answer position concentrates on the entity names.

Code: `https://github.com/CL3-410/individual-project-Seronic2001` (branch `phase-3`). Phase 1 corpora and tokenizers: `https://www.kaggle.com/datasets/shubhadeepmandal/lma-hindi-artifacts` and `https://www.kaggle.com/datasets/shubhadeepmandal/lma-assamese-artifact`. Every number below is read from the result files in `report/results/`, and every figure is drawn from them by `report/make_figures.py`, except the two attention heatmaps, which the `lma-eval-attention` run drew directly (`common/attention_shift.py`); the phase reports give the full detail.

1 Project overview

Phase 1, Data & tokenizers	Phase 2, Pretraining	Phase 3, Reasoning
Hindi 723M and Assamese 528M corpus tokens (estimates); NFC/NFKC normalisation, script filter, OCR repair, SHA-256 and MinHash dedup; 16K BPE with byte fallback.	V1 (Pre-LN, GELU, learned positions) and V2 (RMSNorm, SwiGLU, RoPE); 8 layers, $d_{\text{model}}=384$, ∼25M parameters; 500M tokens, 15,259 steps. Test PPL 36.15 (Hindi), 51.55 (Assamese).	Symbolic-solver synthetic data; disjoint name pools and depth splits; Direct vs CoT SFT with 2 seeds; leakage and attention analysis. Best test accuracy 0.961 (Hindi), 0.833 (Assamese).

The project studies how far small language models, trained from scratch in one language, can be taken: from a raw corpus, through pretraining, to multi-step relational reasoning. Hindi stands for a higher-resource and Assamese for a lower-resource Indic language. The two languages share no data, tokenizers or parameters; every component exists twice, once per language.

2 Phase 1: corpora and tokenizers

Table 1: Corpus and tokenizer statistics. Corpus tokens are the `dataset_stats.json` estimates; training tokens are counted in the token files with each language's 16K BPE tokenizer.

	Hindi (H)	**Assamese (L)**
Corpus tokens (estimate)	723,321,981	528,500,000
Manual collection	148,676,877 (20.55%)	118,800,000 (22.48%)
Training tokens (`train.bin`)	714,015,795	717,422,008
Fertility (tokens/word)	1.2776	1.4313
Characters / token	3.7445	4.5774
`<unk>` rate	0.0%	0.0%

Cleaning applies NFC/NFKC normalisation, keeps only characters of the language's script block, repairs OCR damage (detached matras, broken conjuncts, detached Assamese suffixes), drops short or mostly non-script documents, and removes exact (SHA-256) and near (MinHash, 0.6) duplicates. The Assamese token files were built from a larger set of cleaned sources than its statistics describe, hence 717.4M training tokens against a 528.5M corpus estimate. Because only script characters are kept, the corpora contain no ASCII digits or punctuation.

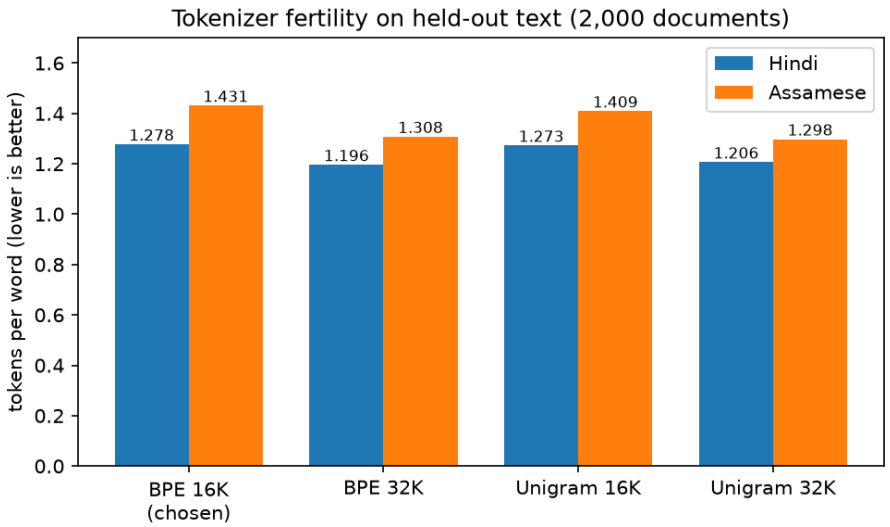


Figure 1: Fertility of the four tokenizer candidates on 2,000 held-out documents. 32K vocabularies are 7–9% more efficient but would leave room for only 6 layers in a 25M-parameter model; BPE and Unigram are within 2% at 16K.

3 Phase 2: pretraining

Both architectures have 8 layers, $d_{\text{model}} = 384$, 6 heads, a 512-token context and tied embeddings (V1: 24,982,272 parameters; V2: 25,172,352). Each was trained for 15,259 AdamW steps of 32,768 tokens (500M tokens, about 0.7 of an epoch) with a cosine schedule from 6×10^{-4} to 6×10^{-5}.

Table 2: Held-out test results on 2,000 non-overlapping 512-token windows (1.02M predicted tokens) per model.

Language	**Model**	**Loss**	**PPL**	**BPB**
Hindi	V1	3.6547	38.66	0.4856
Hindi	**V2**	**3.5876**	**36.15**	**0.4767**
Assamese	V1	3.9987	54.53	0.4634
Assamese	**V2**	**3.9425**	**51.55**	**0.4569**

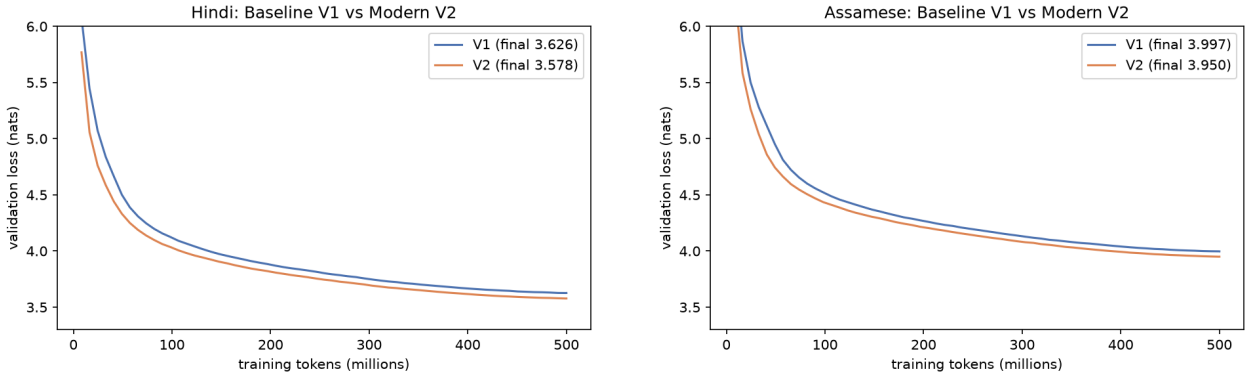


Figure 2: Validation loss of V1 and V2 (left: Hindi, right: Assamese). V2 leads by 0.31 nats early in training and by 0.05 nats at the end.

V2 is better in both languages by 0.06–0.07 nats, an advantage that is constant across context lengths and mostly reflects faster early learning; validation loss was still falling at the last step of every run. Assamese has higher token perplexity but lower bits per byte than Hindi: per unit of text it is not the weaker model. Greedy decoding loops (73–77% repeated 3-grams), nucleus sampling gives the best chrF in all four models, and V1 and V2 generate text of near-identical quality.

4 Phase 3: relational reasoning

4.1 Data and fine-tuning

Synthetic examples state comparisons between named people on one attribute (age, height, savings) and ask about one pair; answers come from a symbolic solver. Train, validation and test names are disjoint, premises are shuffled and stated in random direction, the question's relation word is drawn independently of the answer, and no training query exceeds 3 hops. Models are fine-tuned with a target-only loss, either on the answer (Direct) or on a rationale followed by the answer (CoT), and checkpoints are selected on the relation accuracy of generated validation answers. Five recipes (plain, real-name mix, dropout, short-query curriculum, CoT warm-up) were run with two seeds each: 44 runs.

4.2 Results

Table 3: Relation accuracy of the kept checkpoints. Training covers 1–3 hops; the majority baseline is 0.46–0.53 on every split.

	Direct				CoT
Model	test	4 hops	6 hops	10 hops	test
Hindi V1	**0.961**	0.974	0.960	0.846	0.799
Hindi V2	0.909	0.952	0.922	0.856	0.716
Assamese V1	0.833	0.874	0.726	0.296	0.704
Assamese V2	0.689	0.746	0.742	0.662	0.498

Is it reasoning? Names and prompts never leak between splits, but 90% of main-test premise structures also occur in training, and answering with the most common training answer for a structure scores 0.95. On 4–10-hop, branch and redundant splits no structure occurs in training and this lookup is at chance, yet Hindi Direct models score 0.82–0.99 there: they apply the relation to chains they have never seen.

Weaknesses. Premises about another attribute lower Hindi Direct accuracy from 0.95–0.99 on plain 3-hop chains to 0.73, although the answers name the right pair. On 10-hop chains the weaker models' answers stop naming the queried pair (Hindi V1 CoT names it in 11% of answers), which is

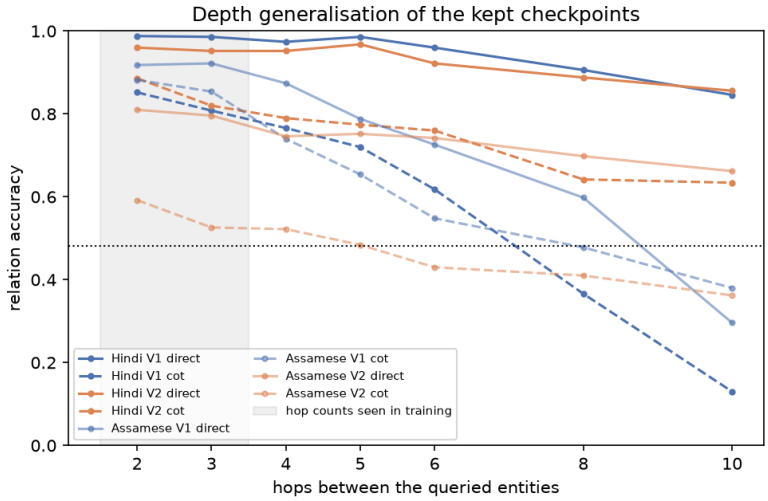


Figure 3: Accuracy against query depth. Direct models (solid) degrade slowly beyond the training depth; CoT models (dashed) degrade quickly.

a failure to carry the chain through generation.

4.3 Attention before and after fine-tuning

Fine-tuning leaves layer-0 attention nearly unchanged (JSD 0.03–0.05) and changes layers 5–7 several times more (Figure 4). In Direct models the last prompt token, where the answer begins, moves from 0.31–0.55 to 0.72–0.99 of its final-layer attention on entity names; CoT models shift less.

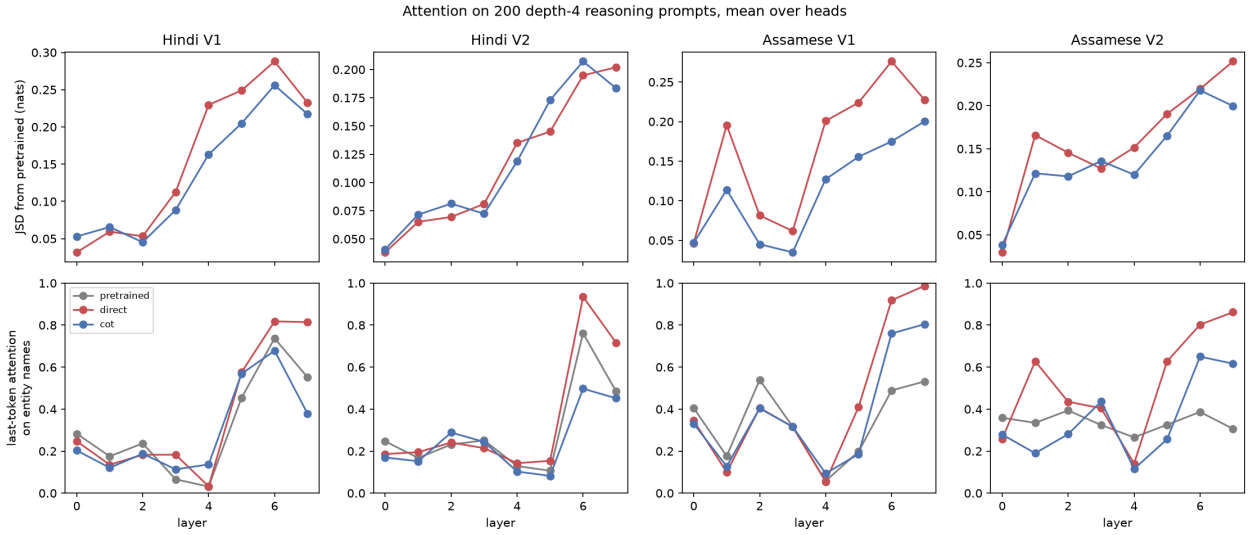


Figure 4: Attention on 200 depth-4 prompts before and after fine-tuning. Top: Jensen–Shannon divergence from the pretrained attention per layer. Bottom: share of the last prompt token's attention on entity-name tokens (grey: pretrained; red: Direct; blue: CoT).

Figure 5 breaks the change down by layer for all eight fine-tuned models. JSD peaks at layer 6 or 7 for every model (0.15–0.29 in layers 5–7 against 0.03–0.05 at layer 0); entropy falls in 60 of the 64 layer–model cells, so attention becomes sharper almost everywhere; and layers 1–5 become more local (36 of 40 cells), while layer 6 attends farther in 7 of the 8 models. Figure 6 isolates the answer position: Direct fine-tuning raises its final-layer attention on entity names in every model (to 0.72–0.99), CoT raises it for Assamese but lowers it for both Hindi models.

Figures 7 and 8 show the same change on a single prompt. In both languages the pretrained last layer spreads its weight thinly over the whole context, apart from a column on the first token; after

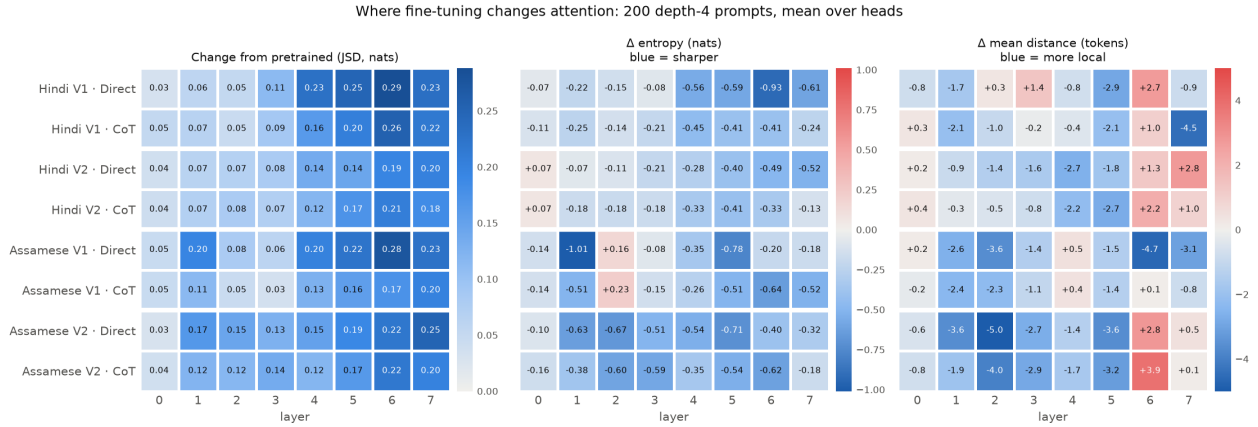


Figure 5: Per-layer change of attention after fine-tuning, for each fine-tuned model (rows) and layer (columns), on 200 depth-4 prompts. Left: Jensen–Shannon divergence from the pretrained model. Middle and right: change in entropy and in mean attention distance (fine-tuned minus pretrained); blue means sharper or more local, red broader or farther.

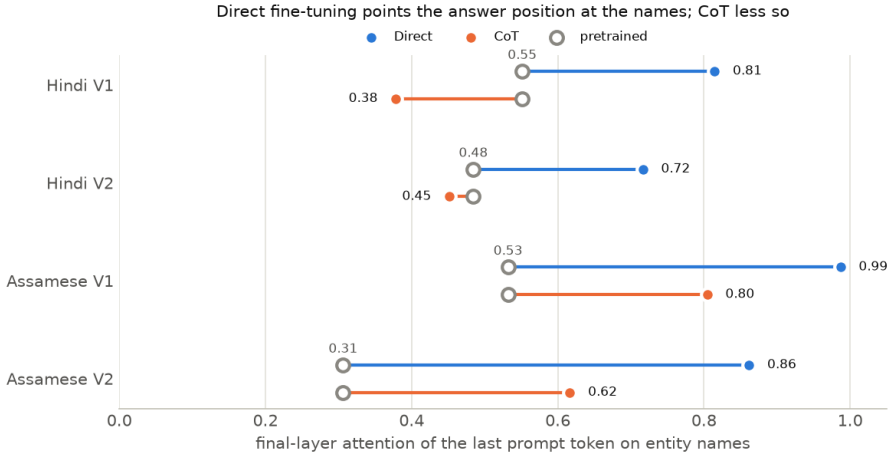


Figure 6: Share of the last prompt token's final-layer attention on entity names: pretrained (hollow) and after Direct (blue) or CoT (orange) fine-tuning.

Direct fine-tuning that first-token column remains, but the diffuse spread gives way to a few strong links per row.

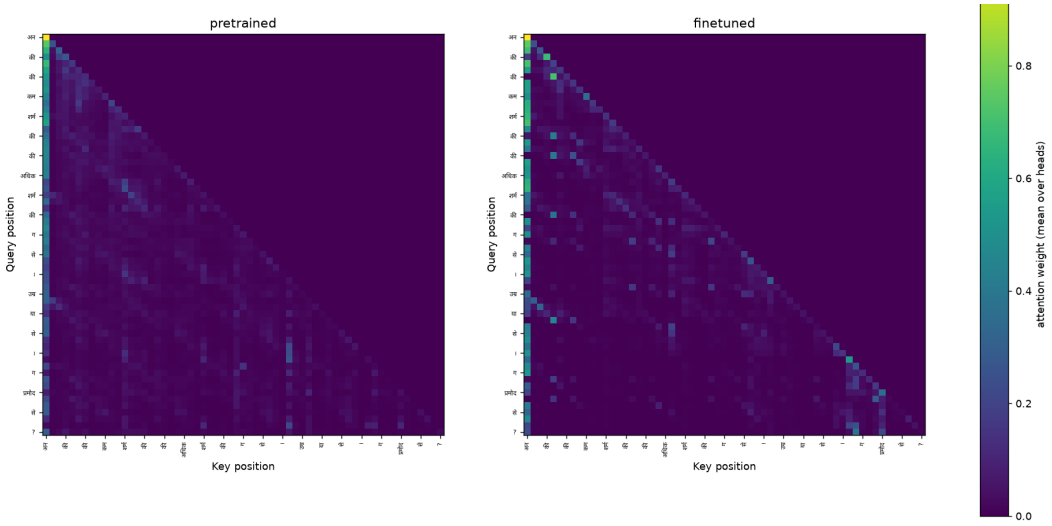


Figure 7: Hindi V1, last layer (mean over heads) on the first depth-4 test prompt: pretrained (left) vs. after Direct fine-tuning (right). Axis labels are the input tokens in order; zoom in to read them.

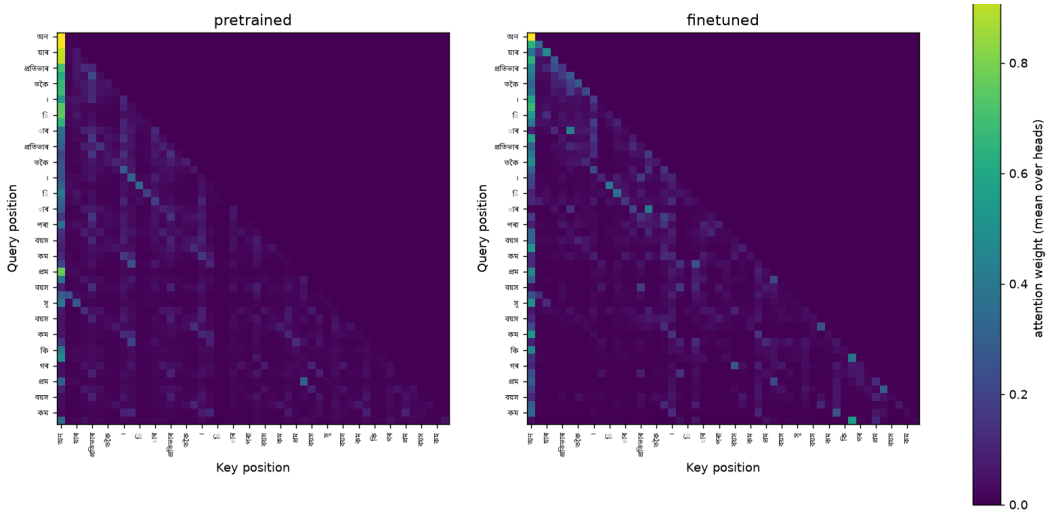


Figure 8: Assamese V1, last layer (mean over heads) on the first depth-4 test prompt: pretrained (left) vs. after Direct fine-tuning (right).

5 The four comparison questions

5.1 Q1. How did data scale and quality differ between Model H and Model L?

Both training sets hold about 715M tokens with their own tokenizers, and both models saw 500M of them. Hindi relies more on large published corpora (Sangraha, OSCAR, CC-100) and crawled literature archives; Assamese more on project crawlers of regional news sites and textbook PDFs. The cleaning pipeline was identical, with extra OCR repairs for Assamese suffixes.

5.2 Q2. How do language-modelling and reasoning results compare across the two tiers?

Assamese has higher token perplexity (51.6 vs 36.2) but lower bits per byte (0.457 vs 0.477), so the two models compress their languages comparably. Reasoning is where the lower-resource language falls behind: 0.833 vs 0.961 on the test set, 0.296 vs 0.846 at 10 hops for V1 Direct, with larger seed variance. A CoT warm-up raised Assamese V1 from 0.655 to 0.831.

5.3 Q3. Which tokenizer and corpus factors most affected the lower-resource model?

Assamese fertility is 12% higher and names inflect by suffixation, so the same entity appears as different token sequences; copying names is a large part of the task (Assamese V2 Direct names the queried pair in 82% of answers, Hindi V1 in 100%). Keeping only script characters also removed ASCII punctuation, so the question mark is a byte-fallback token in both languages.

5.4 Q4. What evidence explains the observed differences?

- Deeper, structurally new splits show generalisation where a lookup fails.
- CoT accuracy falls steadily with depth while Direct stays above 0.84 for Hindi V1: a written rationale compounds errors.
- V2 has the better language model but learns the task less well and copies names less reliably; we hypothesise, without testing it here, that V1's learned absolute positions suit these short templated prompts.
- Fine-tuning re-wires the late layers to attend to the entities the answer depends on.
- Attribute binding is the shared weakness and the natural next step: training data with mixed-attribute premises.

6 Artifacts

Datasets: `lma-hindi-artifacts`, `lma-assamese-artifact` (corpora, token files, tokenizers). **Kaggle runs:** `lma-unify-*` (tokenizers, token files), `lma-pretrain-*` (checkpoints), `lma-eval-pretrained`, `lma-finetune`, `lma-finetune-assamese`, `lma-eval-hard`, `lma-eval-attention`; provenance of each result file in `report/results/README.md`. The earlier `lma-phase2-artifacts`, `lma-phase3-artifacts` and `lma-bonus-artifacts` datasets hold checkpoints of earlier runs, not the models reported here.